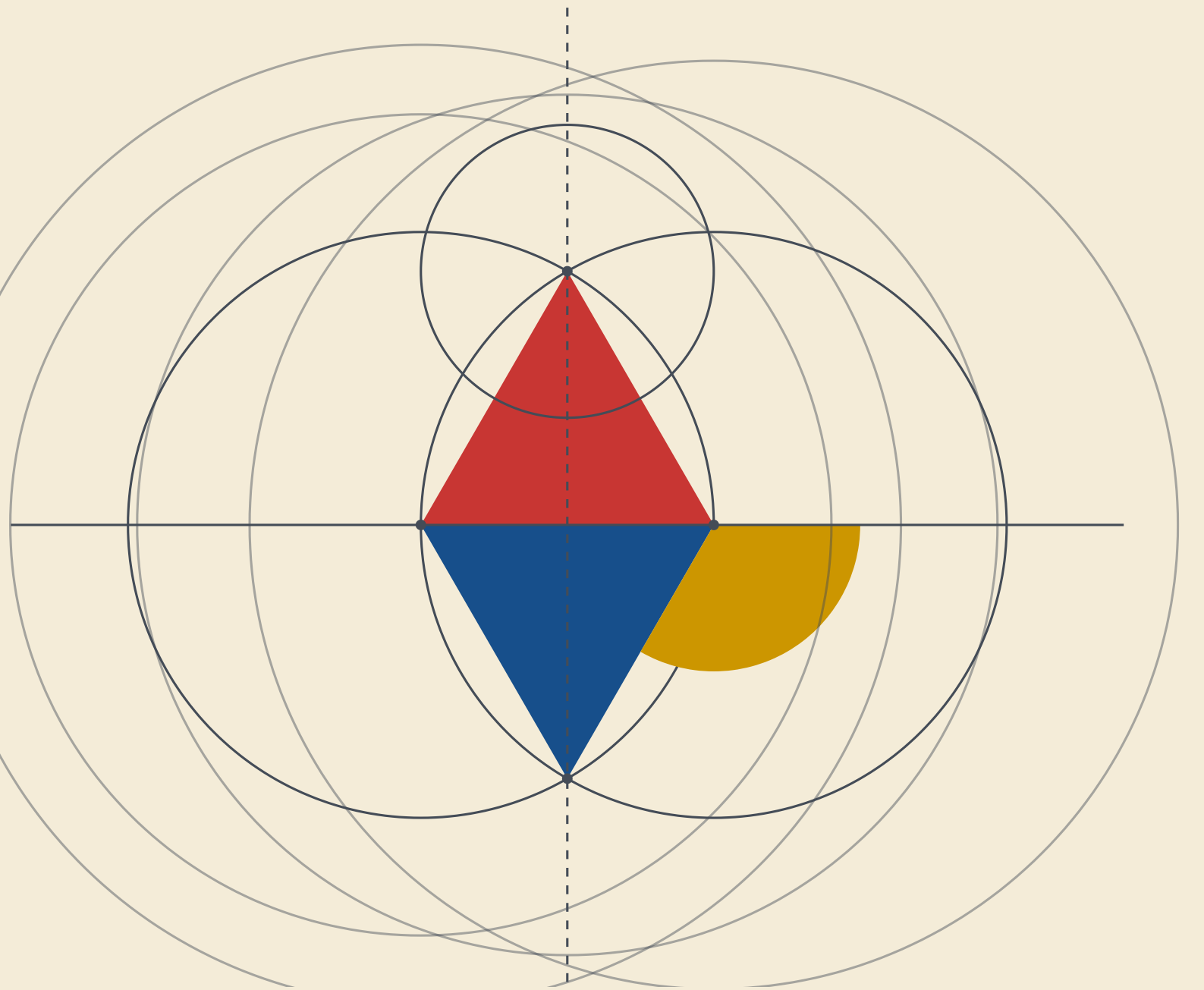


数值方法课程讲义 · 逐节逐行精讲

Tutorial 6 最小二乘回归 · 中文详细讲解版

从胡克定律到正规方程、Python 三法、对数变换、多项式过拟合与 curve_fit 直接非线性拟合——原文全内容逐项精讲

2026年11月2日

目录

导读：这一讲到底在解决什么问题 3

第一章 什么是最小二乘回归 5

第二章 线性代数方法：把拟合看成投影 11

第三章 多元微积分方法：殊途同归 16

第四章 Python 三种最小二乘方法 19

第五章 非线性回归（一）：对数变换「拉直」 23

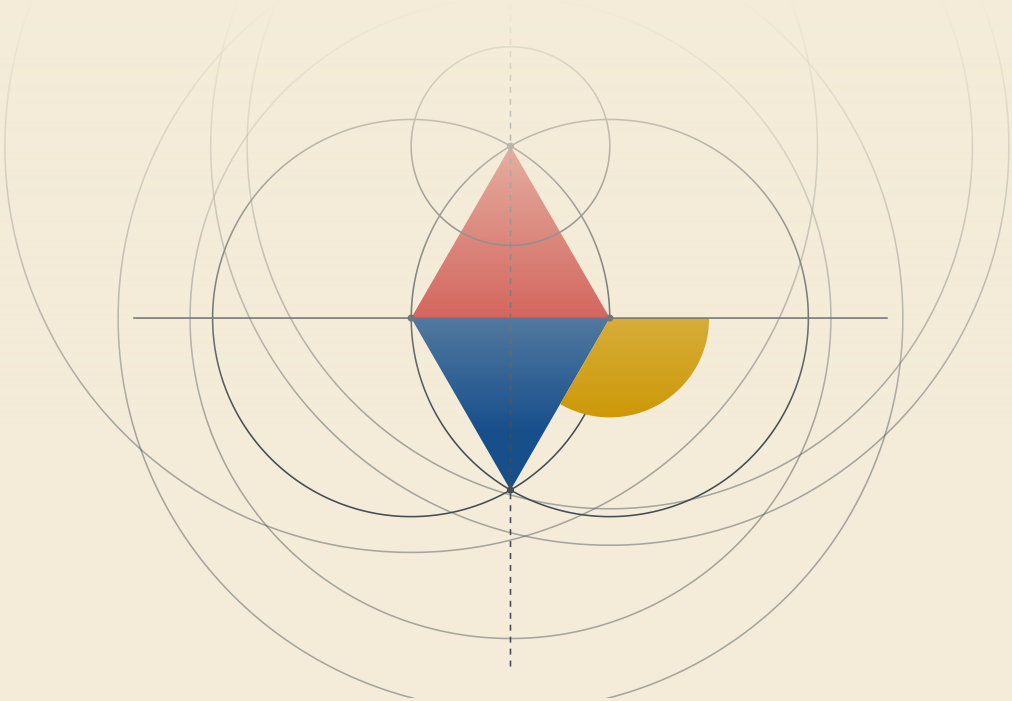
第六章 多项式回归与过拟合 25

第七章 直接非线性拟合：scipy.optimize.curve_fit 29

第八章 全讲总结：方法怎么选 32

附录 A 原文内容覆盖对照表 34

附录 B 符号表 35



导读：这一讲到底在解决什么问题

原文 Tutorial 6 的主题只有一句话：**实验数据永远带噪声，没有一条曲线能同时穿过所有数据点；那就找一条「总体上离所有点最近」的曲线**。「最近」的标准，就是让每个点预测误差的平方之和最小——这就是最小二乘回归（Least Squares Regression）。本讲义把原文的每一节、每一个公式、每一段代码、每一个输出和每一张图全部保留，并逐项翻译、拆解和点评；原文没有展开的推导步骤，本讲义补齐；原文写错的地方，本讲义明确标出并给正确写法（见第三章的勘误框），绝不悄悄改掉。

怎样使用这份讲义

01 **先读每节开头的「原文要点」**
这是原文内容的忠实中文转述，一句不缺

02 **再读公式拆解**
逐个符号说明它是什么、单位是什么、从哪来

03 **然后逐行读代码**
每段原文代码完整保留，并配逐行注释表和输出解读

04 **最后看「易错点」框**
汇总考试与写代码时最容易栽跟头的地方

7	13	4	3
原文大节（H2）总数，本讲义全部覆盖	原文代码与输出单元，逐段保留并讲解	原图，全部原样收录并逐图解读	Python 拟合路线：正规方程、lstsq、curve_fit

全讲地图

章	主题	核心问题
第一章	什么是最小二乘回归	为什么数据对不上模型？误差怎么定义？
第二章	线性代数方法	怎样把拟合写成矩阵问题？正规方程从哪来？
第三章	多元微积分方法	对误差函数求偏导，为什么得到同一个方程？
第四章	经典直线公式	$y = mx + b$ 的斜率、截距闭式解是什么？
第五章	Python 三种方法	同一问题在 NumPy / SciPy 里怎么写？
第六章	对数变换	指数、幂函数怎样「拉直」成直线再拟合？
第七章	多项式与过拟合	阶数越高越好吗？RSS 与 R^2 说明什么？
第八章	curve_fit 直接非线性拟合	拉不直的函数怎么办？误差棒从哪来？
附录	覆盖对照表与符号表	对照原文逐项核对，一项不漏

要点 · 一句话主线

最小二乘的全部故事就是一个优化问题：选参数 $\boldsymbol{\alpha}$，使 $E = \sum_{i=1}^{m}(y_i - \hat{y}(x_i))^2$ 最小。线性代数从「投影」看它，微积分从「梯度为零」看它，两条路最后都走到同一个正规方程 $A^T A\boldsymbol{\beta} = A^T Y$。

第一章 什么是最小二乘回归

1.1 原文要点：参数要从带噪声的数据里估

原文开篇（Welcome to Tutorial 6! Today, we're discussing least squares regression.）指出：在科学与工程实验中，模型参数很少能事先知道（rarely known a priori），必须从实测数据中反推；但真实数据不可避免地含有传感器不准、背景噪声和测量误差。最小二乘回归就是一种数学方法：**在数据点集合中找一条最贴合的曲线或模型，准则是让观测值与模型预测之间差（残差，residuals）的平方和最小。**

> **注 · 「最贴合」必须先定标准**
>
> 「哪条线最好」本来是个含糊的说法。最小二乘把它变成一个可计算的标准：把每个点的竖直偏差平方后加起来，谁的总和最小谁就赢。平方有两个好处：一是正负偏差不会互相抵消，二是函数处处可导、方便求最优解（第三章会用到）。

1.2 物理动机：胡克定律测弹簧刚度

原文用弹簧实验引入。胡克定律：

$$F(x) = kx$$

符号	含义	单位
F	弹簧恢复力（实测）	N （牛顿）
x	偏离平衡位置的位移	cm 或 m
k	弹簧刚度（stiffness）——**待估参数**	如 N/cm

实验室做法：施加一系列已知位移 x_i，测对应力 F_i。由于测量噪声，**没有任何一条直线能穿过全部数据点**（原文原话：no single straight line passes through every data point）。最小二乘要找的，是最能代表这个系统的「最可能」的刚度 k （the most probable stiffness k ）。

提示 · 为什么这个例子好

它的模型只有一个参数 k，且真实答案在仿真里已知（后面代码设 $k=100$）。于是我们可以亲眼看到：估计值不会恰好等于真值，噪声有多大、估计就偏多少——这是理解一切回归的起点。

1.3 估计函数与基函数：什么叫「线性」回归

一般地，最小二乘要在 m 个数据点 $(x_1,y_1),(x_2,y_2),\dots,(x_m,y_m)$ 中找一个估计函数 $\hat{y}(x)$。它通常写成 n 个基函数（basis functions）$f_j(x)$ 的线性组合：

$$\hat{y}(x)=\sum_{j=1}^{n}\alpha_j f_j(x)=\alpha_1 f_1(x)+\alpha_2 f_2(x)+\cdots+\alpha_n f_n(x)$$

原文给出的关键性质，逐条讲解：

原文要点	讲解
α_j **是待求的未知参数（系数）**	它们是我们要「估」的东西；数据点已知，未知的是系数
$f_j(x)$ **是基函数，必须线性无关**	即任何一个都不能写成其余几个的线性组合；否则参数不唯一、矩阵不可逆（第二章）
基函数举例： $f(x)=1$、$f(x)=x$、$f(x)=x^2$、$f(x)=\sin(x)$	常数、一次、二次、正弦都可以当「积木」；选哪些积木决定模型形状
「线性」指对参数 α_j **线性**	哪怕 $f_j(x)=x^2$ 或 $\sin(x)$ 对 x 是弯的，只要 α_j 是以一次形式出现的，仍叫线性回归，求解仍归结为线性方程组

注意 · 本讲第一个高频考点

「线性回归为什么能拟合曲线？」——因为线性指的是**对参数线性**，不是对 x 线性。用 $1,x,x^2$ 做基函数就能拟合抛物线，仍是线性回归（第七章的多项式回归正是如此）。

把估计函数在全部 m 个数据点上取值，得到一个 $m\times n$ 方程组：

$$\begin{aligned}\hat{y}(x_1)&=\alpha_1 f_1(x_1)+\alpha_2 f_2(x_1)+\cdots+\alpha_n f_n(x_1)\\ \hat{y}(x_2)&=\alpha_1 f_1(x_2)+\alpha_2 f_2(x_2)+\cdots+\alpha_n f_n(x_2)\\ &\vdots\\ \hat{y}(x_m)&=\alpha_1 f_1(x_m)+\alpha_2 f_2(x_m)+\cdots+\alpha_n f_n(x_m)\end{aligned}$$

读法：每一行是一个数据点；每行里 $f_j(x_i)$ 都是已知数，未知的只有同一组 $\alpha_1, \ldots, \alpha_n$。通常 $m > n$（点比参数多），方程个数多于未知数，一般无精确解——这正是第二章「超定系统」的由来。

1.4 残差与总平方误差：优化的靶心

每个数据点的个体误差（残差）定义为：

$$e_i = y_i - \hat{y}(x_i)$$

目标是最小化总平方误差 E，它是残差向量 $\mathbf{e} = [e_1, e_2, \ldots, e_m]^T$ 的欧几里得（L_2）范数的平方：

$$E = \|\mathbf{e}\|_2^2 = \sum_{i=1}^{m} e_i^2 = \sum_{i=1}^{m} \big(y_i - \hat{y}(x_i)\big)^2$$

符号对照（原文逐条给出）：m 为数据点总数；y_i 为实际观测的因变量；$\hat{y}(x_i)$ 为模型在 x_i 处的预测值。带帽子 $\hat{}$ 一律表示「估计／预测」，不带帽子的是「实测真值」，这个约定贯穿全讲。

1.5 代码一：先「试」三根候选直线，谁的 E 最小

原文第一段代码不急着求最优解，而是用仿真弹簧数据，手动比较 $k = 70, 100, 130$ 三个候选刚度的总平方误差。完整代码如下（原文原样，仅排版微调）：

```python
import numpy as np
import matplotlib.pyplot as plt

# simulate noisy experimental spring measurements: F = k * x + noise
np.random.seed(42)
x_disp = np.linspace(0.1, 3.0, 30)          # displacement from rest in cm
k_true = 100.0                               # true stiffness = 100 N/cm
noise = np.random.normal(0, 15.0, len(x_disp))
F_meas = k_true * x_disp + noise             # measured force in N

# evaluate total squared error E for different candidate stiffness values
candidate_k = [70.0, 100.0, 130.0]
plt.figure(figsize=(9, 5))
plt.scatter(x_disp, F_meas, color='blue', alpha=0.7, label='Measured Data $(x_i,
F_i)$')

colors = ['green', 'red', 'purple']
for k_cand, col in zip(candidate_k, colors):
    F_pred = k_cand * x_disp
    total_error = np.sum((F_meas - F_pred) ** 2)
    plt.plot(x_disp, F_pred, color=col, linestyle='--', linewidth=2,
             label=f'Candidate $k = {k_cand:.0f}$ N/cm (E = {total_error:.1f}
N$^2$)')
    print(f"Stiffness k = {k_cand:5.1f} N/cm → Total Squared Error E =
{total_error:9.2f} N^2")

plt.title("Hooke's Law: Force vs. Displacement with Candidate Fits", fontsize=13)
plt.xlabel("Displacement from Rest $x$ [cm]", fontsize=11)
plt.ylabel("Force Measured in Spring $F$ [N]", fontsize=11)
plt.legend(loc='upper left', fontsize=10)
plt.show()
```

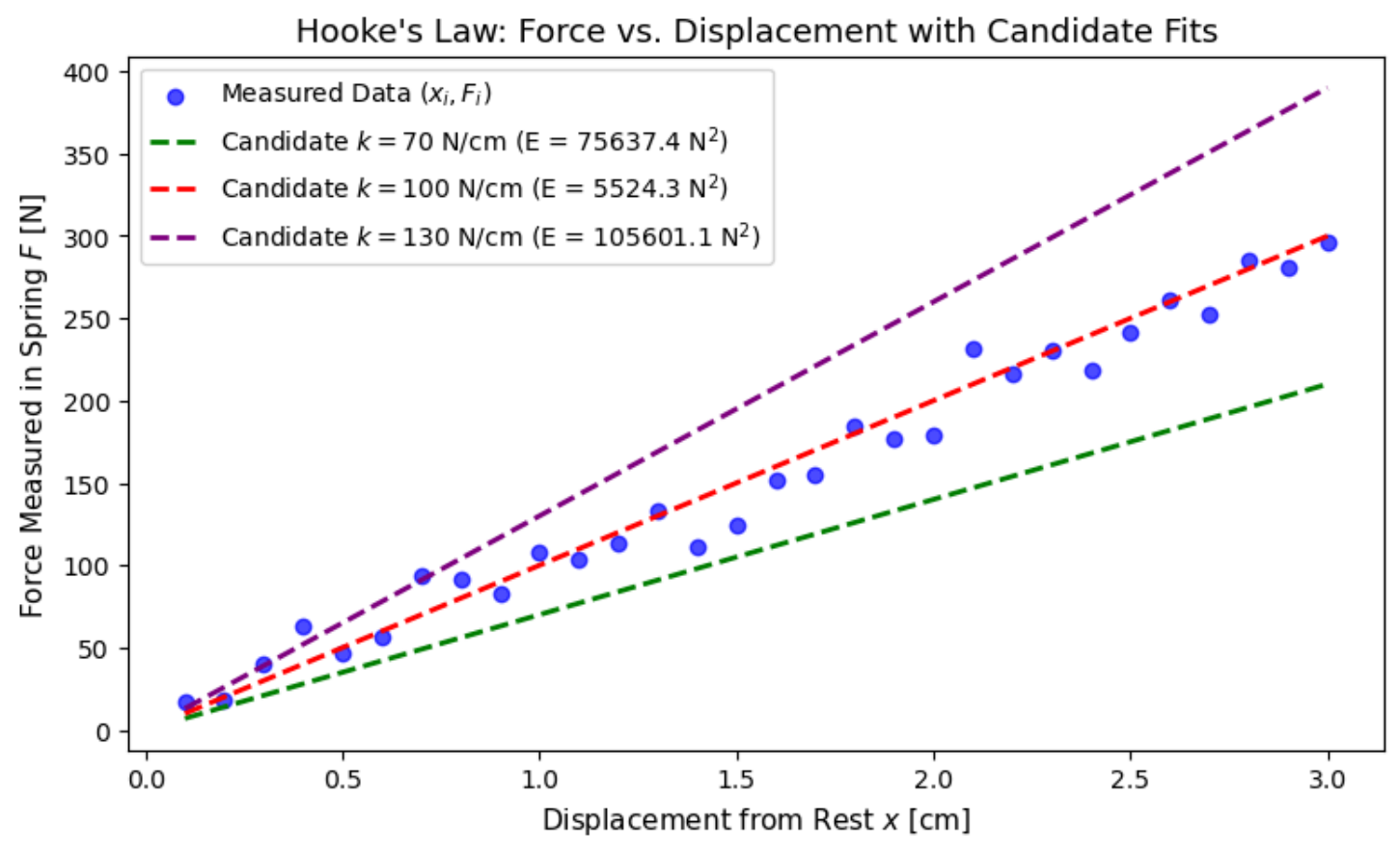
逐行讲解:

代码	做什么	为什么这样写
`np.random.seed(42)`	固定随机种子	保证每次运行噪声完全一样，结果可复现；换种子数值会变
`np.linspace(0.1, 3.0, 30)`	位移从 0.1 到 3.0 cm 等距取 30 个点	模拟实验里「施加已知位移」这一步；这些 x_i 视为无误差
`k_true = 100.0`	真实刚度 100 N/cm	仿真的「标准答案」，真实实验中并不知道，只能靠回归估
`np.random.normal(0, 15.0, ...)`	均值 0、标准差 15 N 的高斯噪声	模拟测量误差；在 $x = 3$ 处信号约 300 N，噪声相对不小
`F_meas = k_true * x_disp + noise`	生成带噪测量力 F_i	这就是实验台上真正读到的数据
`candidate_k = [70, 100, 130]`	三个候选刚度	先不求最优，只比较——呼应「最小二乘就是比谁的 E 小」
`F_pred = k_cand * x_disp`	候选模型预测	注意：此模型 $F = kx$ **没有截距**，直线被钉死穿过原点（位移为零时力为零，符合物理）
`np.sum((F_meas - F_pred) ** 2)`	计算 $E = \sum_i (F_i - \hat{F}_i)^2$	这正是 1.4 节定义的 E，单位是 N^2
图例里带上各自的 E	把数值写进图例	看图时即可直接比较三根线的好坏

原文运行输出：

```text
Stiffness k =  70.0 N/cm → Total Squared Error E =  75637.45 N^2
Stiffness k = 100.0 N/cm → Total Squared Error E =   5524.27 N^2
Stiffness k = 130.0 N/cm → Total Squared Error E = 105601.10 N^2
```

读数：真值 $k = 100$ 的 E 最小（5524.27），$k = 70$ 约为它的 13.7 倍，$k = 130$ 约为 19 倍。这验证了直觉——离真值越远，残差平方和涨得越快（平方放大了偏差）。但请注意：「三**选一**」**里真值获胜，不等于最优解就是 100**；真正的最优 k 要让 E 取到数学上的最小值，第二章的矩阵方法会一步算出它（在带截距的模型下结果是斜率 94.7290，见 2.5 节的说明）。



(x_i, F_i)；绿、红、紫三条虚线分别为 $k = 70, 100, 130$ N/cm 的候选直线，图例标注各自总平方误差 E。红线 $(k = 100)$ 最贴合点云。">

图 1 图 1（原文图）：胡克定律候选拟合。蓝点为带噪测量点 (x_i, F_i)；绿、红、紫三条虚线分别为 $k = 70, 100, 130$ N/cm 的候选直线，图例标注各自总平方误差 E。红线 $(k = 100)$ 最贴合点云。

逐图解读：横轴位移 x （cm），纵轴力 F （N）。点云整体沿一条过原点、斜率约 100 的直线散开，散开的宽度就是标准差 15 N 噪声的直观样子。绿线（$k = 70$）明显偏平，大位移处系统性低于数据；紫线（$k = 130$）过陡；红线（$k = 100$）穿过点云中央。三条线都钉在原点——这是单参数模型 $F = kx$ 的特征，与后面允许截距的模型不同。

第二章 线性代数方法：把拟合看成投影

2.1 三个主角：Y、A、β

原文的线性代数表述是：找最优参数向量 β，使 $A\beta$ 尽可能接近观测向量 Y。三个符号（原文逐条定义）：

观测向量（$m \times 1$ 列向量）：

$$Y = \begin{bmatrix} \\ \\ \\ \end{bmatrix}$$

设计矩阵（design matrix，$m \times n$），每一列是一个基函数在全部数据点上的取值：

$$A = \begin{bmatrix} \\ \\ \\ \end{bmatrix}$$

参数向量（$n \times 1$ 列向量）：

$$\beta = \begin{bmatrix} \\ \\ \end{bmatrix}$$

以最简单的直线 $y = mx + b = \alpha_1 x + \alpha_2$ 为例，基函数是 $f_1(x) = x$ 与 $f_2(x) = 1$，于是：

$$A = \begin{bmatrix} \\ \\ \\ \end{bmatrix}$$

注 · 设计矩阵为什么叫「设计」

因为它由实验设计（在哪些 x_i 处测）和模型设计（选哪些基函数）共同决定。A 的每一行对应一次测量，每一列对应一个参数的「影响通道」。后面代码里 `np.vstack([x, np.ones(len(x))]).T` 造的正是这个 $[x_i \ \ 1]$ 矩阵：第一列是 x，第二列全 1（对应截距）。

全部数据点的模型预测可以紧凑地写成一句话：

$$\hat{Y} = A\beta$$

总平方误差也随之写成向量范数形式：

$$E = \|Y - \hat{Y}\|_2^2 = \|Y - A\beta\|_2^2$$

这一句就是 1.3 节那个 $m \times n$ 方程组的矩阵版：$A\beta$ 的第 i 行恰好是 $\hat{y}(x_i)$。

2.2 几何图像：Y 向列空间投影

关键洞察（原文）：$A\beta$ 是 A 的各列的线性组合，所以它必然落在 A 的列空间（column space / range）$\mathrm{col}(A)$ 里。

当方程组超定（$m > n$，点多于参数）时，噪声使 Y 几乎不可能恰好落在 $\mathrm{col}(A)$ 内——也就是说不存在让 $E = 0$ 的 β。

于是在 $\mathrm{col}(A)$ 中离 Y 欧氏距离 $\|Y - A\beta\|$ 最近的点 $\hat{Y} = A\beta$，正是 Y 在 $\mathrm{col}(A)$ 上的**正交投影**（orthogonal projection）。

> 够不着 Y，就找它正下方的投影点。投影点的标志，是残差与列空间垂直。

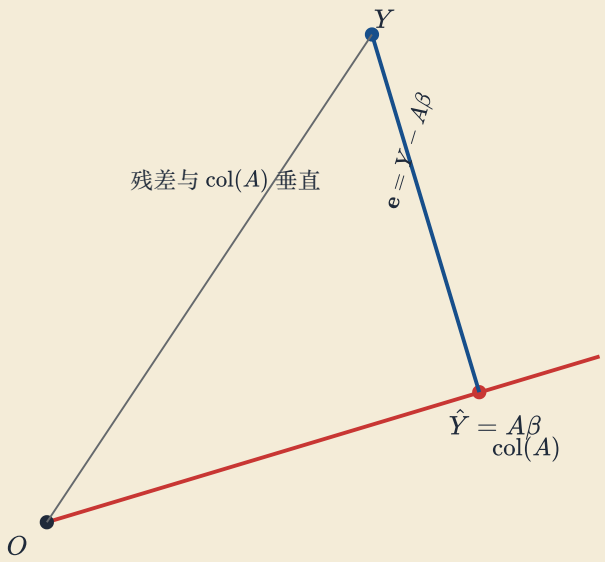


图 2　图 2（本讲义绘制）：最小二乘的几何本质——观测向量 Y 在设计矩阵列空间 col(A) 上的正交投影点 Ŷ=Aβ 距离 Y 最近，残差 e 与列空间垂直。

2.3 正规方程：由「垂直」写出

$\hat{Y}$ 是投影点，等价于残差向量 $\mathbf{e} = Y - A\beta$ 与 A 的每一列都垂直（正交）：

$$A^T(Y - A\beta) = \mathbf{0}$$

原文提醒：两向量点积 $\mathrm{dot}(\mathbf{v}, \mathbf{w}) = \mathbf{v}^T\mathbf{w}$。把上式展开、令每个列方向的点积为零：

$$A^TY - A^TA\beta = \mathbf{0}$$

$$A^TA\beta = A^TY$$

这个方程组就叫**正规方程**（Normal Equations）——本讲最核心的公式。若 A 的各列线性无关（满列秩，full column rank n），则 $n \times n$ 矩阵 A^TA 必可逆，两边乘 $(A^TA)^{-1}$ 得闭式解：

$$\beta = (A^TA)^{-1}A^TY$$

要点 · 正规方程为什么出现 A^TA

A^TA 的 (k, j) 元是 A 第 k 列与第 j 列的点积（列之间有多「像」），A^TY 的第 k 元是第 k 列与观测 Y 的点积（该基函数与数据有多「对路」）。正规方程说的就是：残差与每一列都不相关（点积为零）。

2.4 代码二：用矩阵乘法解弹簧问题

原文紧接着用代码把新模型 $y = \alpha_1 x + \alpha_2$（斜率加截距）套到第一章的同一组弹簧数据上：

```python
# assemble the design matrix A and target vector Y
# model: y = alpha_1 * x + alpha_2  (slope and intercept)
A = np.vstack([x_disp, np.ones(len(x_disp))]).T   # (m, n) = (30, 2)
Y = F_meas[:, np.newaxis]                         # (m, 1) = (30, 1)

# compute normal matrix A^T * A and vector A^T * Y
ATA = np.dot(A.T, A)          # equivalent to A.T @ A
ATY = np.dot(A.T, Y)          # equivalent to A.T @ Y

# direct least squares solution via normal equations: beta = (A^T A)^(-1) A^T Y
beta = np.dot(np.linalg.inv(ATA), ATY)
alpha_1, alpha_2 = beta[0, 0], beta[1, 0]    # indexed like so because it's a
column vector

print(f"\nEstimated Slope (alpha_1 [k]):      {alpha_1:.4f} N/cm")
print(f"Estimated Intercept (alpha_2 [F0]): {alpha_2:.4f} N")
```

逐行讲解：

代码	做什么
`np.vstack([x_disp, np.ones(len(x_disp))]).T`	先把 x 与全 1 两行竖叠成 2×30，再转置成 30×2 的设计矩阵 A：每行 $[x_i,\ 1]$，与 2.1 节公式逐行对应
`F_meas[:, np.newaxis]`	把一维数组 $(30,)$ 升成列向量 $(30, 1)$，与 Y 的 $m \times 1$ 定义对齐；注释特意标了形状，养成查 shape 的习惯
`np.dot(A.T, A)`	算 $A^T A$（2×2）；注释说明等价于 `A.T @ A`，`@` 是 Python 3.5+ 的矩阵乘法运算符
`np.linalg.inv(ATA)`	对 $A^T A$ 求逆；能求逆的前提正是列满秩（截距列全 1 与 x 列不成比例）
`beta[0, 0], beta[1, 0]`	β 是列向量，取第 0、1 行第 0 列得到斜率与截距两个标量

原文输出：

```text
Estimated Slope (alpha_1 [k]):      94.7290 N/cm
Estimated Intercept (alpha_2 [F0]): 5.3478 N
```

2.5 结果解读：为什么斜率不是 100，截距不是 0

- 真值是斜率 100、截距 0 ；估出 94.7290 与 5.3478。偏差来自那 30 个标准差 15 N 的噪声点——最小二乘给出的是「这批数据下最可能」的参数，而不是真值本身；数据越多，估计通常越靠近真值。
- 本节模型比第一章多了截距 α_2（代码打印里记作 F_0）。多一个自由度，模型可以整体上下平移来迁就噪声，所以斜率不必再独自扛起全部拟合，数值自然与第一章「钉死过原点」的候选实验不同——**两个实验模型不同，结果不可直接比大小**，这是读这两节时最容易混淆的一点。
- 截距 5.35 N 相对几百牛的力很小，与「真实截距为 0、只是噪声造成的偏移」的解释相符。

第三章 多元微积分方法：殊途同归

3.1 把 E 看成 n 元函数

原文的第二条推导路线：把总平方误差看成以 n 个参数 $(\alpha_1, \ldots, \alpha_n)$ 为自变量的普通函数：

$$E(\alpha_1, \alpha_2, \ldots, \alpha_n) = \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right)^2$$

符号（原文逐条）：m 数据点数；n 基函数个数；α_j 未知参数；$f_j(x_i)$ 基函数在数据点 x_i 的取值；y_i 实际目标值。求最小值的方法是微积分的老规矩：对每个参数求偏导并令其为零，$\dfrac{\partial E}{\partial \alpha_k} = 0$（$k = 1, \ldots, n$）。

3.2 原文跳过的中间步骤，本讲义补齐

原文只写「After some computation, we get（经过一些计算得到）」。把这一步展开：对 α_k 求偏导时，只有预测式里含 α_k 的那一项 $f_k(x_i)$ 贡献导数，由链式法则：

$$\frac{\partial E}{\partial \alpha_k} = -2 \sum_{i=1}^{m} \left(y_i - \sum_{j=1}^{n} \alpha_j f_j(x_i) \right) f_k(x_i) = 0$$

移项、把求和号拆开，就得到原文的结果式（每个 $k = 1, \ldots, n$ 各一条，共 n 条）：

$$\sum_{j=1}^{n} \alpha_j \left(\sum_{i=1}^{m} f_j(x_i) f_k(x_i) \right) = \sum_{i=1}^{m} f_k(x_i) y_i$$

3.3 与矩阵记号对上号

原文指出（本讲核心对照）：

标量和式	它是什么	矩阵里的位置
$\sum_{i=1}^{m} f_j(x_i) f_k(x_i)$	A 第 k 列与第 j 列的点积	$A^T A$ 的 (k, j) 元
$\sum_{i=1}^{m} f_k(x_i) y_i$	A 第 k 列与 Y 的点积	$A^T Y$ 的第 k 元

于是 n 条标量方程合起来正是：

$$A^T A \boldsymbol{\alpha} = A^T Y, \qquad \boldsymbol{\alpha} = (A^T A)^{-1} A^T Y$$

原文结论：**线性代数（投影）与多元微积分（梯度为零）两条路给出完全相同的最小二乘解**。这不奇怪：投影条件「残差与每列垂直」$A^T(Y - A\beta) = \mathbf{0}$ 与梯度条件 $\nabla E = \mathbf{0}$ 本来就是同一件事的两种语言。考试若问「正规方程的两种推导」，答出「几何投影」与「偏导置零」即满分骨架。

3.4 经典直线模型 $y = mx + b$ 的闭式公式

两参数直线是 $n = 2$ 的特例。原文写：令 $\partial E/\partial m = 0$ 与 $\partial E/\partial b = 0$，得斜率与截距的闭式解。

注意 · 原文公式有一处符号冲突（重要勘误，考试别照抄）

原文印作 $m = \dfrac{m\sum x_i y_i - \sum x_i \sum y_i}{m\sum x_i^2 - (\sum x_i)^2}$、$b = \dfrac{\sum y_i - m\sum x_i}{m} = \bar{y} - m\bar{x}$。同一个字母 m 被同时当作**斜率**和**数据点个数**（后文代码里点数其实叫 `m_points`），直接代入会自相矛盾。正确写法是把点数记作 N、斜率记作 s：

$$s = \frac{N\sum x_i y_i - \sum x_i \sum y_i}{N\sum x_i^2 - (\sum x_i)^2}, \qquad b = \frac{\sum y_i - s\sum x_i}{N} = \bar{y} - s\bar{x}$$

其中 $\bar{x}, \bar{y}$ 是样本均值；$b = \bar{y} - s\bar{x}$ 还说明：**最优直线必过数据重心** $(\bar{x}, \bar{y})$。紧随其后的原文代码用的正是正确版本（`m_points` 当点数、`slope_calc` 当斜率），所以数值结果无误——错的只是这一行公式的字母复用。

原文代码（用求和公式算同一组弹簧数据）：

```python
# compute slope m and intercept b using the formulas above
m_points = len(x_disp)
sum_x  = np.sum(x_disp)
sum_y  = np.sum(F_meas)
sum_xy = np.sum(x_disp * F_meas)
sum_x2 = np.sum(x_disp ** 2)

slope_calc = (m_points * sum_xy - sum_x * sum_y) / (m_points * sum_x2 - sum_x ** 2)
intercept_calc = (sum_y - slope_calc * sum_x) / m_points

print(f"Slope m [k]:      {slope_calc:.4f} N/cm")
print(f"Intercept b [F0]: {intercept_calc:.4f} N")

# matches what we got with the linear algebra approach!
```

逐行看：先备齐四个求和 $\sum x_i$、$\sum y_i$、$\sum x_i y_i$、$\sum x_i^2$ 与点数 `m_points`，再套勘误框里的正确公式。原文输出：

```text
Slope m [k]:      94.7290 N/cm
Intercept b [F0]: 5.3478 N
```

与第二章矩阵法**逐字相同**（94.7290 / 5.3478）——这正是代码注释「matches what we got with the linear algebra approach!」的含义，也是 3.3 节「殊途同归」在数值上的当场验证。

第四章 Python 三种最小二乘方法

4.1 同一问题，三种写法

原文在新数据上并排演示三种实现。先造数据：$y = 1 + x + x \cdot$ 噪声，x 在 $[0, 1]$ 上取 101 个点：

```python
from scipy import optimize

# generate data: y = 1 + x + (x * noise)
x = np.linspace(0, 1, 101)
np.random.seed(1)
y = 1 + x + x * np.random.random(len(x))

# assemble matrix A and turn y into a column vector
A = np.vstack([x, np.ones(len(x))]).T
y_col = y[:, np.newaxis]

# Method 1: direct least squares regression using normal equations
alpha_direct = np.dot((np.dot(np.linalg.inv(np.dot(A.T, A)), A.T)), y_col)
print("Method 1 (Direct Normal Equation):")
print(alpha_direct)

# Method 2: Use numpy.linalg.lstsq
alpha_lstsq = np.linalg.lstsq(A, y_col)[0]
print("\nMethod 2 (numpy.linalg.lstsq):")
print(alpha_lstsq)

# Method 3: Use optimize.curve_fit from scipy (no need to turn y into a column
vector)
def linear_func(x, a, b):
    return a * x + b

alpha_scipy = optimize.curve_fit(linear_func, xdata=x, ydata=y)[0]
print("\nMethod 3 (scipy.optimize.curve_fit):")
print(alpha_scipy)

# results
plt.figure(figsize=(8, 5))
plt.plot(x, y, 'b.', label='Generated Data Points')
plt.plot(x, alpha_direct[0] * x + alpha_direct[1], 'r-', linewidth=2,
         label=f'Fitted Line: $\hat{{y}} = {alpha_direct[0,0]:.2f}x +
{alpha_direct[1,0]:.2f}$')
plt.xlabel('x', fontsize=11)
plt.ylabel('y', fontsize=11)
plt.title('Least Squares Linear Regression in Python', fontsize=13)
plt.legend(loc='upper left', fontsize=10)
plt.show()
```

数据生成要注意：噪声项是 $x \cdot U(0,1)$（`np.random.random` 是 $[0,1)$ 均匀分布），**噪声幅度随 x 增大而增大**（异方差），且恒为正——所以拟合参数不会恰好是 $(1,1)$。真实关系 $y = 1 + x$ 只是噪声的中心线。

4.2 三种方法逐项对比

方法	核心调用	返回形状	特点与适用
方法 1：正规方程手算	`inv(A.T@A)@A.T@y_col`	列向量 $[[a],[b]]$	与第二章公式逐字对应，教学最直观；但显式求逆在数值上最不稳健（A^TA 会放大条件数），实际工程少这样写
方法 2：`np.linalg.lstsq`	`np.linalg.lstsq(A, y_col)[0]`	列向量 $[[a],[b]]$	基于 SVD，不显式求逆，数值最稳；返回元组的第 $[0]$ 个才是系数，后面还有残差、秩、奇异值。**线性问题首选**
方法 3：`scipy.optimize.curve_fit`	先定义 `linear_func(x,a,b)`，再 `curve_fit(...)[0]`	一维数组 $[a,\ b]$	要先把模型写成函数；不需要把 y 转列向量（代码注释特别说明）；同一接口能处理非线性（第七章），且能给参数误差

原文输出（三种方法数值完全一致）：

```text
Method 1 (Direct Normal Equation):
[[1.54120263]
 [0.98370554]]

Method 2 (numpy.linalg.lstsq):
[[1.54120263]
 [0.98370554]]

Method 3 (scipy.optimize.curve_fit):
[1.54120263 0.98370554]
```

读数：斜率 1.5412、截距 0.9837。截距接近真值 1 ，斜率明显大于 1 ——因为噪声 $xU(0,1)$ 的期望是 $0.5x$，数据的期望线其实是 $1+1.5x$，所以斜率估到 1.54 恰恰合理；这提醒我们：**先想清噪声加在哪、以什么形式加，再评价参数估得「准不准」**。

4.3 那段 SyntaxWarning 是什么

原文还保留了运行警告（完整收录）：

```text
<>:34: SyntaxWarning: invalid escape sequence '\h'
<>:34: SyntaxWarning: invalid escape sequence '\h'
/tmp/ipykernel_123/223895980.py:34: SyntaxWarning: invalid escape sequence '\h'
```

讲解：第 34 行图例字符串里的 $\hat{y}$ 写在普通字符串中，反斜杠加 h 不是合法转义序列（合法的如换行、制表符），Python 于是警告 `invalid escape sequence`；它目前仍按字面输出所以图没坏，但新版本可能报错。修法任选其一：字符串前加 `r` 变原始字符串，或把反斜杠写双。警告重复出现三行，是 Jupyter 内核多通道回显所致，不是代码跑了三遍。



生成的数据，红线为拟合直线 $\hat{y} = 1.54x + 0.98$。点云呈喇叭口——越往右散得越开，正是噪声正比于 x 的样子。">

图 3 图 3（原文图，本讲编号；原文顺序第 2 张）：Python 三种方法的拟合结果。蓝点为 $y = 1 + x + x \cdot U(0,1)$ 生成的数据，红线为拟合直线 $\hat{y} = 1.54x + 0.98$。点云呈喇叭口——越往右散得越开，正是噪声正比于 x 的样子。

第五章 非线性回归（一）：对数变换「拉直」

5.1 思路：变形成直线，再套线性最小二乘

原文指出：有些物理关系在原始形式下不是基函数的线性组合，但**取对数可以把问题等价地变成线性形式**（logarithmic transformations）。套路固定四步：取对数、认出直线、对变换后的数据做普通线性最小二乘、把斜率截距反变换回原参数。

5.2 指数函数 $y = \alpha e^{\beta x}$

模型（$\alpha > 0$ 为尺度因子 scale factor，β 为速率常数 rate constant）：

$$y = \alpha e^{\beta x}$$

两边取自然对数 $\ln$，用 $\ln(ab) = \ln a + \ln b$、$\ln e^{\beta x} = \beta x$：

$$\ln(y) = \ln\left(\alpha e^{\beta x}\right) = \ln(\alpha) + \beta x$$

令 $\tilde{y} = \ln(y)$、$c = \ln(\alpha)$，立即得到一条直线：

$$\tilde{y} = \beta x + c$$

对数据 $(x_i, \tilde{y}_i)$ 做标准线性最小二乘，得斜率与截距后反变换：

$$\beta = \text{slope}, \qquad \alpha = e^{c}$$

注意只有 y 被变换、x 原样保留——图上叫「半对数图」（$\ln y$ 对 x 仍是直线）。

5.3 幂函数 $y = bx^{m}$

模型（$b > 0$，且必须 $x > 0$ 才能取对数）：

$$y = bx^{m}$$

两边取自然对数，用 $\ln x^{m} = m \ln x$：

$$\ln(y) = \ln\left(bx^{m}\right) = m\ln(x) + \ln(b)$$

令 $\tilde{y} = \ln(y)$、$\tilde{x} = \ln(x)$、$c = \ln(b)$：

$$\tilde{y} = m\tilde{x} + c$$

这次 x 、y 双双变换（双对数图上是直线）。线性拟合后反变换：

$$m = \text{slope}, \qquad b = e^{c}$$

5.4 两类变换对照与注意事项

模型	变换	变换后直线	斜率给谁	截距反变换
$y = \alpha e^{\beta x}$	只对 y 取 ln	$\ln y = \beta x + \ln \alpha$	β	$\alpha = e^{c}$
$y = bx^{m}$	对 x, y 都取 ln	$\ln y = m \ln x + \ln b$	指数 m	$b = e^{c}$

注意 · 对数变换的三个前提（原文条件加读图常识）

第一，取 $\ln y$ 要求所有 $y_i > 0$，取 $\ln x$ 要求 $x_i > 0$（原文对幂函数明写 $x > 0$）；有零或负值就不能直接变换。第二，变换后最小二乘最小化的是**对数空间**的误差，对应原始空间近似相对误差，大 y 点的权重会被压低——拟合目标变了，结果与第七章的直接拟合不会完全相同。第三，反变换只动截距相关的参数（$\alpha = e^{c}$、$b = e^{c}$），斜率（β 或 m）直接照用，别也套指数。

第六章 多项式回归与过拟合

6.1 模型与 np.polyfit

n 次多项式模型（原文定义）：

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_2 x^2 + a_1 x + a_0$$

它正是 1.3 节「基函数取 $1, x, x^2, \ldots, x^n$」的特例——对参数线性，所以仍是线性回归。NumPy 专用函数（原文）：`np.polyfit(x, y, deg)` 计算 `deg` 次最小二乘多项式系数。补充两个读代码必需的约定：`polyfit` 返回的系数**按次数从高到低排列**（a_n 在最前）；`np.polyval(coeffs, x)` 按这些系数在给定 x 处求多项式的值。

6.2 欠拟合与过拟合（原文定义）

- **欠拟合（Underfitting）**：低阶多项式（如对弯曲数据用 1 次直线）抓不住真实物理趋势。
- **过拟合（Overfitting）**：高阶多项式灵活性过剩，能几乎穿过每个训练点（训练误差近零），却在点与点之间剧烈振荡，对新数据的泛化（generalize）极差。

6.3 代码三：1 到 6 次多项式逐个试

数据来自原文注明的讲义幻灯（Lecture slides PDF 2, Slide 5），9 个点：

```python
# Dataset from lecture slides (PDF 2, Slide 5)
x_d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8])
y_d = np.array([0, 0.8, 0.9, 0.1, -0.6, -0.8, -1, -0.9, -0.4])

plt.figure(figsize=(12, 8))
x_dense = np.linspace(0, 8, 200)

for i in range(1, 7):
    # get the polynomial coefficients
    y_est = np.polyfit(x_d, y_d, i)

    # evaluate goodness of fit
    y_fitted = np.polyval(y_est, x_d)   # evaluates the polynomial at given x
values
    rss = np.sum((y_d - y_fitted) ** 2)

    plt.subplot(2, 3, i)
    plt.plot(x_d, y_d, 'o', markersize=6, label='Data')
    # evaluate the values for a smooth polynomial curve
    plt.plot(x_dense, np.polyval(y_est, x_dense), 'r-', linewidth=1.5,
label='Fit')
    plt.title(f'Polynomial order {i} (RSS={rss:.3f})', fontsize=11)
    plt.xlabel('x', fontsize=9)
    plt.ylabel('y', fontsize=9)

plt.tight_layout()
plt.show()

# summary table of polynomial fits
print("Polynomial Fit Summary:")
for order in range(1, 7):
    coeffs = np.polyfit(x_d, y_d, order)
    residuals = y_d - np.polyval(coeffs, x_d)
    rss = np.sum(residuals ** 2)
    r2 = 1.0 - rss / np.sum((y_d - np.mean(y_d)) ** 2)
    print(f"Order {order}: RSS = {rss:7.4f}, R^2 = {r2:.4f}")
```

逐行要点：`x_dense` 在 $[0,8]$ 取 200 个密点画平滑曲线（模型线），而 RSS 只在 9 个真实数据点上算（`y_fitted` 用 `x_d`）；`plt.subplot(2, 3, i)` 把六次尝试排成 2 行 3 列对照图，每幅标题写明阶数与 RSS；第二个循环重算并打印汇总表，其中 $R^2 = 1 - \mathrm{RSS}/\sum(y_d - \bar{y}_d)^2$，分母是总平方和 TSS（数据自身的总波动），所以 R^2 表示「模型解释了百分之多少的波动」。

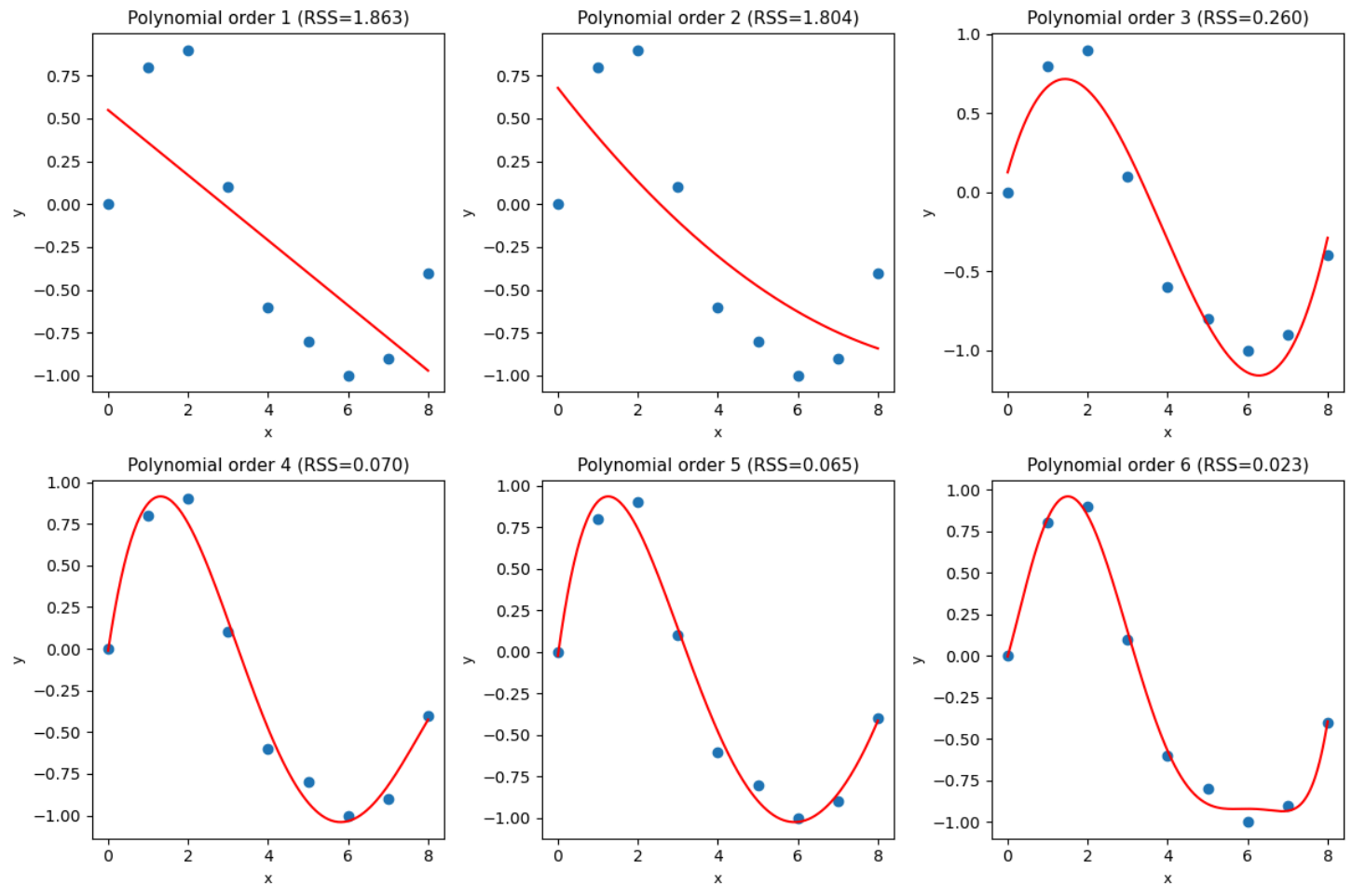


图 4　图 4（原文图，本讲编号；原文顺序第 3 张）：1–6 次多项式拟合对照（2×3 子图）。圆圈为 9 个数据点，红线为各阶拟合曲线，标题标注 RSS。低阶线条僵硬、高阶曲线在点间起伏加剧，是过拟合的经典画像。

逐阶读图：1 次（RSS 1.863）一条斜直线，完全跟不上数据先升后降再回升的弯——欠拟合；2 次（1.804）略弯仍差；3 次（0.260）抓住主体形状，RSS 骤降；4 次（0.070）更贴；5 次（0.065）与 6 次（0.023）在训练点上近乎全穿，但曲线在数据稀疏处开始蛇形摆动——这就是原文警告的「点间振荡、泛化差」。

6.4 汇总表：RSS 只会越小，R^2 只会越大

原文输出（完整）：

```text
Polynomial Fit Summary:
Order 1: RSS =  1.8629, R^2 = 0.5376
Order 2: RSS =  1.8038, R^2 = 0.5523
Order 3: RSS =  0.2595, R^2 = 0.9356
Order 4: RSS =  0.0696, R^2 = 0.9827
Order 5: RSS =  0.0654, R^2 = 0.9838
Order 6: RSS =  0.0227, R^2 = 0.9944
```

表 1　各阶多项式的残差平方和与决定系数（原文输出数据）

阶数	RSS	R^2	解读
1	1.8629	0.5376	欠拟合：只解释约 54% 波动
2	1.8038	0.5523	几乎没改善，弯度仍不够
3	0.2595	0.9356	跃升点：主体趋势被抓住
4	0.0696	0.9827	继续变好，边际收益变小
5	0.0654	0.9838	与 4 阶几乎并列
6	0.0227	0.9944	训练指标最好，但最可能过拟合

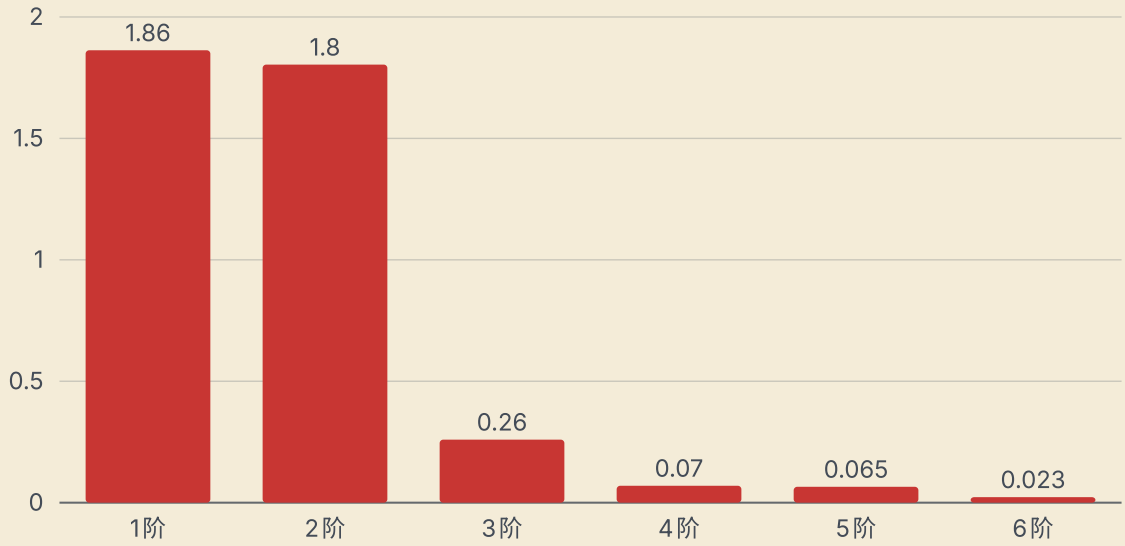


图 5　图 5（本讲义绘制）：RSS 随多项式阶数单调下降——这由数学保证（参数越多，训练误差只会更小），因此不能只凭 RSS 或 R^2 选阶数。

注意 · 本章考点：训练指标好不等于模型好

阶数升高时旧模型是新模型的子集（把高次系数设为 0 即回到低阶），所以训练 RSS 必然单调不增、R^2 必然单调不减——6 阶「最好」是数学必然，不代表它最接近真实规律。判断过拟合要看**新数据**上的表现（验证集、交叉验证），或结合图像看曲线是否在点间不合理地振荡。本例只有 9 个点，6 阶已有 7 个参数，自由度所剩无几，更应警惕。

第七章 直接非线性拟合：scipy.optimize.curve_fit

7.1 什么时候对数变换救不了

原文举例：双指数和 $y = a_1 e^{-k_1 x} + a_2 e^{-k_2 x}$、频率未知的正弦函数等，**无法线性化**（cannot be linearised）——取对数拆不开求和，参数也不能分离成「斜率加截距」。这时只能直接非线性回归，主力工具就是 `scipy.optimize.curve_fit`。

原文使用要点：它采用迭代法（iterative method），因此**最好给初值猜测**（initial guess）；不给时 SciPy 默认每个参数初值都是 1。函数返回两样东西：

返回值	含义
`popt`	最优参数数组（optimal parameter values）
`pcov`	`popt` 的估计协方差矩阵；对角元开根号 $\sqrt{\mathrm{pcov}[j,j]}$ 是第 j 个参数的**标准误差（不确定度）**

为什么需要初值：线性最小二乘有闭式解、一步到位；非线性问题的 E 曲面可能有多个低谷，迭代从初值出发下坡，初值太离谱可能走进错误的低谷或不收敛。这也是它与前面方法最本质的区别。

7.2 代码四：直接拟合指数函数

```python
# generate exponential data with noise
x = np.linspace(0.1, 10, 50)
np.random.seed(42)
alpha_true, beta_true = 0.124, 0.28
y = alpha_true * np.exp(beta_true * x) + np.random.normal(0, 0.05, len(x))

# define the function form
def func(x, a, b):
    y = a * np.exp(b * x)
    return y

# direct nonlinear fitting with curve_fit
popt, pcov = optimize.curve_fit(func, xdata=x, ydata=y, p0=[0.1, 0.2])
alpha_fit, beta_fit = popt[0], popt[1]
alpha_err, beta_err = np.sqrt(pcov[0, 0]), np.sqrt(pcov[1, 1])

print(f"Fitted parameters: alpha = {alpha_fit:.6f} +/- {alpha_err:.6f}")
print(f"                   beta  = {beta_fit:.6f} +/- {beta_err:.6f}")

# plot the data and fit
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b.', label='Noisy Experimental Data')
plt.plot(x, func(x, alpha_fit, beta_fit), 'r-', linewidth=2,
         label=f'Direct Fit: $y = {alpha_fit:.3f} e^{{{beta_fit:.3f}x}}$')
plt.xlabel('x', fontsize=11)
plt.ylabel('y', fontsize=11)
plt.title('Direct Nonlinear Fitting with scipy.optimize.curve_fit', fontsize=13)
plt.legend(loc='upper left', fontsize=11)
plt.show()
```

逐行要点：真值 $\alpha = 0.124$、$\beta = 0.28$，噪声是标准差 0.05 的高斯噪声**加在原始 y 上**（与第五章对数法隐含处理的误差结构不同，结果不会完全一致）；`func(x, a, b)` 的签名必须是「自变量在前、参数在后」，`curve_fit` 靠这个约定识别参数；`p0=[0.1, 0.2]` 给出离真值不远的初值；误差由 `np.sqrt(pcov[j,j])` 取对角元开根号，正是 7.1 节原文公式的代码实现；图例里三重花括号，前两层是给 LaTeX 留的上标括号、内层才是 f-string 字段。

原文输出：

```text
Fitted parameters: alpha = 0.119231 +/- 0.004634
                   beta  = 0.282969 +/- 0.004556
```

读数：α 估值 0.119231 与真值 0.124 相差 0.0048，约为其标准误 0.0046 的 1 倍；β 估值 0.282969 与真值 0.28 相差 0.0030，约为标准误的 0.65 倍——两个真值都落在「估值加减约 1 个标准误」范围内，说明拟合与不确定度报告自洽。这正是 `pcov` 的价值：**不仅给参数，还给每个参数的误差棒**，报告结果时应写成 $\alpha = 0.1192 \pm 0.0046$ 这种形式。

已达 2 左右，噪声点在曲线上下均匀散布。">

图 6　图 6（原文图，本讲编号；原文顺序第 4 张）：curve_fit 直接非线性拟合。蓝点为指数真值加高斯噪声的数据，红线为直接拟合曲线。注意纵轴随指数增长，右端 y 已达 2 左右，噪声点在曲线上下均匀散布。

第八章 全讲总结：方法怎么选

8.1 三条路线一张表

表 2　最小二乘方法选型总表

情形	方法	工具	要点
模型对参数线性	解正规方程 $A^TA\beta = A^TY$	`np.linalg.lstsq`（首选，SVD 稳健）或手算正规方程	设计矩阵 A 列满秩才可逆；直线闭式公式是 $n=2$ 特例
指数 $y=\alpha e^{\beta x}$、幂 $y=bx^m$	先对数变换拉直，再线性拟合	线性最小二乘加反变换 $\alpha=e^c$ 等	要求 $y>0$（幂函数还需 $x>0$）；拟合的是对数空间误差
多项式趋势未知	`np.polyfit` 逐阶试	看图加验证数据选阶	RSS 与 R^2 随阶数必然变好，不能单独作选阶依据
拉不直的非线性（双指数、未知频率正弦等）	直接迭代拟合	`scipy.optimize.curve_fit`，给 `p0` 初值	返回 `popt` 与 `pcov`，对角开根号即参数标准误差

8.2 公式速查

公式	用途
$e_i = y_i - \hat{y}(x_i)$，$E = \sum_i e_i^2 = \|Y - A\beta\|_2^2$	残差与总平方误差，一切方法的靶心
$\hat{y}(x) = \sum_j \alpha_j f_j(x)$	估计函数：基函数的线性组合，「线性」指对参数线性
$A^TA\beta = A^TY$，$\beta = (A^TA)^{-1}A^TY$	正规方程与闭式解（投影与梯度两路同归）
$s = \dfrac{N\sum x_iy_i - \sum x_i \sum y_i}{N\sum x_i^2 - (\sum x_i)^2}$，$b = \bar{y} - s\bar{x}$	直线斜率截距（勘误后正确版）；最优直线过重心 $(\bar{x}, \bar{y})$
$\ln y = \beta x + \ln\alpha$；$\ln y = m\ln x + \ln b$	指数／幂函数的对数线性化
$R^2 = 1 - \mathrm{RSS}/\sum(y_i - \bar{y})^2$	决定系数：解释的波动占比
$\mathrm{SE}_j = \sqrt{\mathrm{pcov}[j,j]}$	curve_fit 参数标准误差

8.3 易错点清单（对照原文逐项）

01 **把「线性」误解为直线**

线性指对参数线性；x^2、$\sin x$ 做基函数仍是线性回归

02 **照抄原文经典公式时不查符号**

原文该行字母 m 一符两义，必须把点数改记 N（第三章勘误框）

03 **混淆两个弹簧实验**

第一章候选线无截距、过原点；第二章模型带截距，斜率 94.7290 不可与候选 E 直接比

04 **显式求逆当成最佳实践**

`inv(A^TA)` 只适合教学演示；实际优先 `lstsq`

05 **忽略 SyntaxWarning**

字符串里写 LaTeX 要用原始字符串或双写反斜杠

06 **对数变换忘查正负**

$\ln y$、$\ln x$ 要求正数；且变换改变了误差权重

07 **只看 RSS 与 R^2 选多项式阶数**

两者随阶数必然单调变好，选阶要看新数据与曲线形状

08 **curve_fit 不给初值**

默认全 1 可能收敛差；返回的 `pcov` 对角开根号才是参数误差，别把整个矩阵当误差

要点 · 最后一句

最小二乘不是一条公式，而是一个习惯：**先写清模型与误差的定义，再选与之匹配的求解器，最后用不确定度和新数据检验结果。** 本讲的弹簧、直线、多项式与指数四组实验，练的都是这同一个习惯。

附录 A 原文内容覆盖对照表

下表按原文（tut-6.html）出现顺序逐项对账，确认每一项都已在本讲义中出现并讲解。

表 3　原文逐项覆盖对账

原文顺序与内容	本讲义位置
标题 TUTORIAL 6 与欢迎语	第一章 1.1
「什么是最小二乘回归」定义段（参数未知、数据噪声、残差平方和最小）	第一章 1.1
胡克定律 $F(x) = kx$ 与 F, x, k 三符号说明	第一章 1.2
估计函数、基函数线性无关、「线性指对参数」三条性质与 $m \times n$ 方程组	第一章 1.3
残差 e_i 、E 定义及 $m, y_i, \hat{y}(x_i)$ 说明	第一章 1.4
代码一：弹簧仿真加三候选 k 比较，及三行 E 输出与图	第一章 1.5 与图 1
线性代数设定：Y, A, β 三定义、直线设计矩阵、$\hat{Y} = A\beta$、E 范数式	第二章 2.1
列空间、正交投影、垂直条件、正规方程与闭式解	第二章 2.2–2.3 与图 2
代码二：设计矩阵与正规方程手算，输出 94.7290 / 5.3478	第二章 2.4–2.5
微积分法：E 函数、偏导置零、和式与矩阵元对应、两路同解结论	第三章 3.1–3.3
经典直线公式（原文符号冲突，已勘误）及求和代码、相同输出	第三章 3.4
Python 三方法代码、三组相同输出、SyntaxWarning	第四章 4.1–4.3 与图 3
对数变换引言；指数与幂函数两小节全部公式与反变换	第五章 5.1–5.4
多项式模型、polyfit、欠拟合与过拟合定义	第六章 6.1–6.2
多项式代码（含数据出处注）、对照图、RSS 与 R^2 汇总六行输出	第六章 6.3–6.4、图 4–5
curve_fit 引言（不可线性化举例、初值、popt 与 pcov 说明）	第七章 7.1
指数直接拟合代码、参数加误差输出、拟合图	第七章 7.2 与图 6

附录 B 符号表

符号	读法与含义
x_i, y_i	第 i 个数据点的自变量与实测因变量
$\hat{y}(x_i)$、$\hat{Y}$	模型预测值（帽子表示估计）；向量版为 $\hat{Y}$
e_i、$\mathbf{e}$	残差 $y_i - \hat{y}(x_i)$ 及其向量
E	总平方误差 $\sum_i e_i^2$，优化目标
m、n	数据点数、基函数（参数）个数；原文经典公式处 m 另有冲突，见勘误
$f_j(x)$	第 j 个基函数
α_j、$\boldsymbol{\beta}$	待估参数及其列向量写法
A	设计矩阵，$m \times n$，行对应数据点、列对应基函数
$\mathrm{col}(A)$	A 的列空间，最优预测 $\hat{Y}$ 所在的集合
A^TA、A^TY	正规方程的系数矩阵与右端项
RSS、R^2	残差平方和；决定系数 $1 - \mathrm{RSS}/\mathrm{TSS}$
`popt`、`pcov`	curve_fit 的最优参数与协方差矩阵